

Relatório Técnico Final: Projeto JardimJá

1. Introdução

Este relatório descreve o trabalho desenvolvido no projeto final da disciplina, o "JardimJá". A ideia do sistema é ser uma plataforma para conectar pessoas que precisam de serviços de jardinagem com prestadores de serviço dessa área. Para fazer isso funcionar bem, dividi o projeto em três partes principais: um backend (`jardimja-backend`), um aplicativo para quem contrata (`jardimja-cliente`) e outro aplicativo focado no trabalhador (`jardimja-prestador`).

O foco deste documento é explicar como montei a arquitetura do projeto, por que tomei certas decisões de design e mostrar os problemas que enfrentei durante o semestre. Além disso, fiz uma reflexão de como os conceitos que vi na teoria, como Arquitetura Orientada a Eventos (EDA), Message-Oriented Middleware (MOM), Clean Architecture e REST, foram aplicados na prática.

2. Descrição da Arquitetura Implementada

Decidi usar um modelo cliente-servidor distribuído. Como eu tinha que fazer dois aplicativos móveis diferentes, o melhor caminho era centralizar a regra de negócio em uma única API.

2.1 Visão Geral

A arquitetura funciona assim: os dois aplicativos mobile se comunicam com o backend através de requisições HTTP, seguindo o padrão REST. Para a maior parte das coisas (como fazer login ou buscar serviços), a comunicação é síncrona. Porém, para algumas operações mais pesadas e demoradas, como enviar notificações para vários prestadores de que um serviço novo surgiu, usei um barramento de mensagens (MOM), aplicando um pouco de EDA (Arquitetura Orientada a Eventos).

2.2 Componentes Principais

- **Backend (`jardimja-backend`):** É o coração do sistema. Ele expõe os endpoints da API REST, faz a validação das regras de negócio, salva tudo no banco de dados e gerencia a fila de mensagens.
- **App do Cliente (`jardimja-cliente`):** Feito em Flutter. É por aqui que o cliente se cadastra, procura jardineiros e pede orçamentos. Escolhi Flutter [\[1\]](#) porque queria compilar para Android e iOS sem ter que fazer o código duas vezes, aproveitando os conceitos de frameworks híbridos [\[2\]](#).
- **App do Prestador (`jardimja-prestador`):** Também em Flutter, mas com telas e fluxos totalmente diferentes. O foco aqui é o prestador receber os alertas de serviço, aceitar as ofertas e ver quanto já ganhou.

3. Decisões de Design

Durante o desenvolvimento, tive que tomar algumas decisões importantes para não deixar o código virar uma bagunça conforme o projeto crescia.

3.1 O uso da Clean Architecture

Tentei seguir a Clean Architecture no frontend (Flutter) e no backend. No começo, parecia que eu estava criando arquivo demais à toa, mas ajudou muito. Separei o código em camadas: a UI (telas), o domínio (regras de negócio) e os dados (repositórios e consumo da API). O maior benefício que percebi foi que, se eu precisasse mudar algo na tela, não quebrava a lógica que faz as chamadas para o backend.

3.2 Dois apps separados

Cheguei a pensar em fazer um aplicativo só, onde a pessoa escolhia se era "Cliente" ou "Prestador" no login. Mas decidi fazer dois projetos separados no Flutter. A jornada do cliente é muito de navegação e busca, enquanto a do prestador é mais focada em receber notificações e gerenciar agenda. Separar evitou um código muito misturado e deixou cada app mais leve.

3.3 Comunicação e Integração

Optei por usar REST em quase tudo. Os verbos HTTP (GET, POST, PUT, DELETE) deixaram claro o que cada endpoint faz. Mas, para processos assíncronos, percebi que precisava de uma abordagem diferente para não travar a API, o que levou à escolha do MOM para as notificações.

4. Dificuldades Encontradas e Soluções Adotadas

Fazer tudo funcionar junto não foi tão fácil quanto parecia no diagrama. Tive alguns gargalos que deram trabalho.

4.1 Gerenciamento de Estado no Flutter

O problema: No começo do desenvolvimento das telas, eu passava os dados de um lado para o outro via construtor (prop drilling). Quando a tela do prestador ficou complexa (com listas de serviços carregando e notificações chegando), o app começou a bugar e mostrar informações erradas.

A solução: Adotei o padrão Provider/Bloc para separar o estado da interface visual. Assim, a tela só escuta as mudanças do domínio e se atualiza sozinha, o que limpou bastante o código.

4.2 Sincronismo no Backend

O problema: Toda vez que um cliente pedia um serviço, a API demorava muito para responder. Isso acontecia porque o backend estava tentando achar os prestadores próximos e mandar notificação para todos eles antes de devolver o status 200 OK para o cliente.

A solução: Aqui entrou a teoria da disciplina [\[3. Aula 11\]](#). Mudei a rota para apenas salvar o pedido no banco e jogar uma mensagem em uma fila (MOM). O backend responde para o cliente na hora. Enquanto isso, um processo em background lê a fila e manda as notificações aos poucos, sem travar nada.

5. Reflexão sobre os Padrões Estudados

Conseguir aplicar o que vi nas aulas teóricas no meu projeto prático fez bastante diferença para entender o porquê desses padrões existirem.

5.1 REST

O uso de REST [\[4\]](#) foi essencial no projeto. Como dividi as partes de front e back, manter os contratos (JSON) bem definidos me permitiu trabalhar em cada parte sem gerar quebras constantes. Além disso, entender o conceito de "Stateless" ajudou a não guardar a sessão do usuário no servidor, usando apenas os tokens JWT.

5.2 Clean Architecture

Como comentei nas decisões de design, a Clean Architecture parece muita burocracia no início, mas força a manter as regras de negócio isoladas. O maior aprendizado foi ver que a interface do usuário (Flutter) e o banco de dados (no backend) são apenas detalhes de infraestrutura; o núcleo do software tem que funcionar independente deles.

5.3 EDA e MOM

Essa foi a parte mais legal de ver funcionando. Os conceitos de Arquitetura Orientada a Eventos e Message-Oriented Middleware [\[3. Aula 11\]](#) mudaram a minha visão de como lidar com gargalos de processamento. Desacoplar a requisição principal do envio de notificações aumentou a resiliência do sistema. Se o serviço de notificação cair, a mensagem fica lá na fila esperando para ser processada depois, ou seja, nenhum dado é perdido.

6. Conclusão

Esse projeto final foi um desafio, mas consolidou muito do que estudei. Construir um sistema com banco de dados, API e dois aplicativos consumindo esses dados mostrou na prática como o baixo acoplamento e a alta coesão são vitais. A utilização da Clean Architecture, REST e as soluções com mensageria (MOM) provaram que organizar bem a arquitetura no início me salvou de muita refatoração e bugs no final.

7. Referências Bibliográficas

- 1. Aula 07 - Flutter. Material de apoio da disciplina DAMD.**
- 2. Aula 08 - Frameworks Híbridos. Material de apoio da disciplina DAMD.**
- 3. Aula 11 - Middleware e MOM (Message-Oriented Middleware). Material de apoio da disciplina DAMD.**
- 4. Aula 12 - REST, Docker e Kubernetes. Material de apoio da disciplina DAMD.**